Week 08 – S2 PRACTICE: Sequelize Advanced, Models & Querying

Goal: Go beyond the basics — add computed instance methods to your models, filter data with advanced query operators, model Many-to-Many relationships (with and without extra fields), and understand the difference between lazy loading and eager loading.

One single project, exercise by exercise. Unlike Week 07, every exercise in this practice builds the same application: a small Library Management System. You will create the project once in Exercise 1, then keep adding files and features to it — by the time you reach Exercise 9 you will have a working REST API reusing everything from Exercises 1–8. Each exercise tells you exactly which file(s) to create or edit.

At the end of this practice, you should be able to...

- Add computed/business logic to a model using the prototype pattern and the `class extends Model` syntax.
- Filter query results with Sequelize operators (`Op.gt` , `Op.between` , `Op.in` , ...) and combine several conditions.
- Declare a Many-to-Many relationship with `belongsToMany` , including a junction table with its own extra columns.
- Explain the difference between lazy loading (`get...()`) and eager loading (`include`), and pick the right one to avoid the N+1 query problem.

How to start:

1. Complete the **Project Setup** section right below first — it scaffolds the folder structure, installs the packages, and creates `config/database.js` .
2. Fill in the blank code blocks (`_____`) with the correct code, in the exact file indicated by the 📁 box at the top of each exercise.
3. Submit the whole project folder on the MS Teams assignment.

Resources:

- Official Sequelize docs — *https://sequelize.org/docs/v6*
- Sequelize with Node.js & MySQL — DigitalOcean tutorial
- Video: Sequelize crash course — YouTube

- Video: Sequelize associations — YouTube
- Video: Sequelize relationships — YouTube

Project Structure

This is the full and final structure of the project — you won't create all of it at once. Each exercise below tells you which lines/files are added at that step. Files are annotated with the exercise that introduces them.

```
library-app/
│
├── config/
│   └── database.js            ← given in the start code (reuse from Week 07)
│
├── models/
│   ├── index.js                ← central place for ALL associations (Ex 5, 6, 8)
│   ├── LibraryMember.js         ← Ex 1  (instance methods)
│   ├── LibraryBook.js           ← Ex 2  (class syntax + query fields)
│   ├── Loan.js                   ← Ex 6  (junction: borrowedAt, returnedAt)
│   └── Review.js                 ← Ex 8  (junction: rating)
│
├── scripts/                      ← small runnable files, one per warm-up exercise
│   ├── ex1-instance-methods.js
│   ├── ex3-query-operators.js
│   ├── ex4-combined-operators.js
│   ├── ex5-favorites.js
│   ├── ex6-loans.js
│   ├── ex7-lazy-eager.js
│   └── ex8-averages.js
│
├── controllers/
│   └── libraryController.js     ← Ex 9  (business logic for the API)
│
├── routes/
│   └── libraryRoutes.js         ← Ex 9  (route definitions)
│
├── .env                          ← DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, PORT
├── server.js                     ← Ex 9  (Express app, mounts routes, calls sync())
├── package.json                  ← "type": "module"
└── README.md
```

Setup – Create the Project & Connect to the Database Warm-up

Goal: Scaffold the `library-app` project, install the packages you'll need for the whole practice, and verify the database connection — before touching any model.

📁 **Create: package.json, .env, config/database.js, scripts/ex0-test-connection.js**

Create the project folder and initialise it:

```
mkdir library-app
cd library-app
npm init -y
mkdir config models scripts controllers routes
```

Install the packages. Fill in the blank:

```
npm install express sequelize mysql2 dotenv
```

Enable ES modules — add this to `package.json`:

```
{
  "type": "module"
}
```

Create `.env` at the project root and fill in your own database settings:

```
DB_HOST=localhost
DB_USER=root
DB_PASS=your_password
DB_NAME=library_app_db
PORT=4000
```

Create a database named `library_app_db` using the MySQL monitor:

```
mysql -u root -p
CREATE DATABASE library_app_db;
```

Create `config/database.js` and fill in the blanks:

```
import { Sequelize } from 'sequelize';
import dotenv from 'dotenv';
dotenv.config();

const sequelize = new Sequelize(
  process.env.DB_NAME,        // DB_NAME
  process.env.DB_USER,
  process.env.DB_PASS,
  {
    host: process.env.DB_HOST,    // DB_HOST
    dialect: 'mysql',          // mysql
    logging: false,
  }
);
export default sequelize;
```

Create `scripts/ex0-test-connection.js` to verify everything works. Fill in the blank:

```
import sequelize from '../config/database.js';
try {
  await sequelize.authenticate();        // authenticate()
  console.log('Database connected successfully!');
} catch (err) {
  console.error('Connection failed:', err);
}
```

You're set! Every exercise from here on reuses this same config/database.js — you will never need to touch it again. Move on to Exercise 1.

Part A — Warm-ups · Building the Library Models

Exercise 1 – Instance Methods: the LibraryMember Model

Goal: Kick off the project by creating the first model, `LibraryMember`, and add a custom method to its prototype so instances can compute derived data.

📁 **Create: models/LibraryMember.js and scripts/ex1-instance-methods.js**

In `models/LibraryMember.js`, fill in the blanks:

```
import { DataTypes } from 'sequelize';
import sequelize from '../config/database.js';

const LibraryMember = sequelize.define('LibraryMember', {          // .define()
  firstName:          DataTypes.STRING,
  lastName:           DataTypes.STRING,
  booksBorrowedCount: { type: DataTypes.INTEGER, defaultValue: 0 },
});

// Add an instance method on the model's prototype
LibraryMember.prototype.getFullName = function () {              // .prototype
  return `${this.firstName} ${this.lastName}`;                   // this.lastName
};

// Add a second instance method: true if booksBorrowedCount >= 5
LibraryMember.prototype.isFrequentReader = function () {
  return this.booksBorrowedCount >= 5;                     // >=
};

export default LibraryMember;
```

In `scripts/ex1-instance-methods.js`, use the computed methods:

```
import sequelize from '../config/database.js';
import LibraryMember from '../models/LibraryMember.js';

await sequelize.sync({ force: true });

const member = await LibraryMember.create({                    // .create()
  firstName: 'Sokha',
  lastName: 'Chan',
  booksBorrowedCount: 7,
});

console.log(member.getFullName());          // getFullName()  ->  "Sokha Chan"
console.log(member.isFrequentReader()); // -> true
```

Exercise 2 – Class Syntax: the LibraryBook Model

Goal: Create the second model, `LibraryBook`, using the class syntax (`extends Model`) instead of `sequelize.define()`.

📁 **Create: models/LibraryBook.js**

Fill in the blanks in `models/LibraryBook.js`:

```
import { DataTypes, Model } from 'sequelize';
import sequelize from '../config/database.js';

class LibraryBook extends Model {                // Model
  getAgeInYears() {
    return new Date().getFullYear() - this.publishedYear;
  }

  // true if pages > 400
  isLongRead() {
    return this.pages > 400;                     // >
  }
}

LibraryBook.init(                                // .init()
  {
    title:         { type: DataTypes.STRING, allowNull: false },
    author:        { type: DataTypes.STRING, allowNull: false },
    publishedYear: DataTypes.INTEGER,
    pages:         DataTypes.INTEGER,
    price:         { type: DataTypes.FLOAT, defaultValue: 0 },
    stock:         { type: DataTypes.INTEGER, defaultValue: 0 },
    genre:         DataTypes.ENUM('fiction', 'non-fiction', 'sci-fi', 'biography')
  },
  { sequelize, modelName: 'LibraryBook' }                    // 'LibraryBook'
);

export default LibraryBook;
```

Quickly verify it (append to ex1 or a new script):

```
import LibraryBook from '../models/LibraryBook.js';

const book = await LibraryBook.create({
  title: 'Dune', author: 'Frank Herbert', publishedYear: 1965, pages: 412,
  price: 25, stock: 3, genre: 'sci-fi',
});
console.log(book.getAgeInYears());  // e.g. 61
console.log(book.isLongRead());         // isLongRead() -> true
```

Exercise 3 – Querying LibraryBook with Operators

Goal: Use Sequelize's `Op` operators to translate conditions like "greater than" or "between".

📁 **Create: scripts/ex3-query-operators.js** | 📌 **Reuses: models/LibraryBook.js**

In `scripts/ex3-query-operators.js`, fill in the blanks:

```
import { Op } from 'sequelize';
import sequelize from '../config/database.js';
import LibraryBook from '../models/LibraryBook.js';

await sequelize.sync({ force: true });
await LibraryBook.bulkCreate([
  { title: 'Book A', author: 'A', price: 25, stock: 5, genre: 'fiction' },
  { title: 'Book B', author: 'B', price: 15, stock: 0, genre: 'sci-fi' },
  { title: 'Book C', author: 'C', price: 50, stock: 2, genre: 'non-fiction' },
  { title: 'Book D', author: 'D', price: 10, stock: 10, genre: 'biography' },
  { title: 'Book E', author: 'E', price: 30, stock: 1, genre: 'fiction' },
  { title: 'Book F', author: 'F', price: 5, stock: 0, genre: 'sci-fi' }
]);

// 1. Books that cost strictly more than $20
const expensive = await LibraryBook.findAll({
  where: { price: { [Op.gt]: 20 } },   // gt
});

// 2. Books that cost $20 or less
const affordable = await LibraryBook.findAll({
  where: { price: { [Op.lte]: 20 } },   // lte
});

// 3. Books with a price between $10 and $30 (inclusive)
const midRange = await LibraryBook.findAll({
  where: { price: { [Op.between]: [10, 30] } },   // 10, 30
});

// 4. Books that are out of stock
const outOfStock = await LibraryBook.findAll({
  where: { stock: 0 },   // 0
});
```

Exercise 4 – Combining Multiple Operators

Goal: Combine several conditions in a single where clause, including `Op.in`.

📁 **Create: scripts/ex4-combined-operators.js**

Find every book whose price is between 10 and 40, genre is fiction or sci-fi, and stock >= 1:

```
const results = await LibraryBook.findAll({
  where: {
     price: { [Op.between]: [10, 40] },
     genre: { [Op.in]: ['fiction', 'sci-fi'] },   // in
     stock: { [Op.gte]: 1 },                      // 1
  }
});
```

Exercise 5 – Many-to-Many Basics: Favorite Books

Goal: Connect LibraryMember and LibraryBook with a simple Many-to-Many relationship (auto-generated junction table).

📁 **Create: models/index.js, scripts/ex5-favorites.js**

In `models/index.js`, declare the association:

```
import sequelize from '../config/database.js';
import LibraryMember from './LibraryMember.js';
import LibraryBook from './LibraryBook.js';

// Sequelize will create a junction table automatically
LibraryMember.belongsToMany(LibraryBook, { through: 'Favorites' });   // belongsToMany
LibraryBook.belongsToMany(LibraryMember, { through: 'Favorites' });   // belongsToMany

export { sequelize, LibraryMember, LibraryBook };
```

In `scripts/ex5-favorites.js`, create sample data:

```
import { sequelize, LibraryMember, LibraryBook } from '../models/index.js';
await sequelize.sync({ force: true });

const sokha = await LibraryMember.create({ firstName: 'Sokha', lastName: 'Chan' });
const dune  = await LibraryBook.create({ title: 'Dune', author: 'Frank Herbert', pub

// .addLibraryBook() is generated by belongsToMany
await sokha.addLibraryBook(dune);          // addLibraryBook(dune)
const favorites = await sokha.getLibraryBooks();          // getLibraryBooks()
```

Exercise 6 – Many-to-Many with Extra Fields: Loans

Goal: Create a junction model `Loan` to track when a book was borrowed/returned.

📁 **Create: models/Loan.js, scripts/ex6-loans.js** | ✏️ **Edit: models/index.js**

Create `models/Loan.js`:

```
import { DataTypes } from 'sequelize';
import sequelize from '../config/database.js';

const Loan = sequelize.define('Loan', {
  borrowedAt: DataTypes.DATEONLY,
  returnedAt: { type: DataTypes.DATEONLY, allowNull: true },   // true (nullable until returned)
});
export default Loan;
```

Edit `models/index.js`:

```
import Loan from './Loan.js';

LibraryMember.belongsToMany(LibraryBook, { through: Loan, as: 'BorrowedBooks' });   // Loan
LibraryBook.belongsToMany(LibraryMember, { through: Loan, as: 'Borrowers' });

export { sequelize, LibraryMember, LibraryBook, Loan };
```

In `scripts/ex6-loans.js`:

```
await sokha.addBorrowedBooks(dune, {
  through: { borrowedAt: '2025-06-17', returnedAt: null },   // through
});

const borrowed = await sokha.getBorrowedBooks();
console.log(`${borrowed[0].title} borrowed on ${borrowed[0].Loan.borrowedAt}`);   // Loan
```

Part B — Core · Querying & the Full API

Exercise 7 – Lazy Loading vs Eager Loading Core

Goal: Diagnose and fix the N+1 query problem.

📁 **Create: scripts/ex7-lazy-eager.js**

Q1 – Spot the Problem

Code:
`const members = await LibraryMember.findAll(); for(const member of members) { await member.getBorrowedBooks(); }`

How many SQL queries run if there are 20 members?	21 (1 to fetch members, + 20 to fetch books for each member individually).
Why is this inefficient?	It sends a flood of tiny queries to the database instead of one or two large ones, causing network overhead (the N+1 problem).

Q2 – Fix It with Eager Loading

```
const members = await LibraryMember.findAll({
  include: { model: LibraryBook, as: 'BorrowedBooks' },   // include
});

for (const member of members) {
  const books = member.BorrowedBooks;    // .BorrowedBooks
  console.log(`${member.firstName}:`, books.map(b => b.title));
}
```

Q3 – Filter on the Included Model

```
const duneReaders = await LibraryMember.findAll({
  include: {
    model: LibraryBook,                          // LibraryBook
    as: 'BorrowedBooks',
    where: { title: 'Dune' },                // 'Dune'
  },
});
```

Exercise 8 – Ratings & Averages Core

Goal: Add a `Review` junction model and calculate average ratings.

📁 **Create: models/Review.js, scripts/ex8-averages.js** | ✏️ **Edit: models/index.js**

Q1 – Define Review and Association

Create `models/Review.js` (rating 0-5 FLOAT). Edit `models/index.js`:

```
import Review from './Review.js';

LibraryMember.belongsToMany(LibraryBook, { through: Review, as: 'ReviewedBooks' });
LibraryBook.belongsToMany(LibraryMember, { through: Review, as: 'Reviewers' });

export { sequelize, LibraryMember, LibraryBook, Loan, Review };
```

Q3 – Compute the Averages (in scripts/ex8-averages.js)

```
// Average rating per book
const books = await LibraryBook.findAll({
  include: { model: LibraryMember, as: 'Reviewers' },   // 'Reviewers'
});

for (const book of books) {
  let sum = 0;
  for (const reviewer of book.Reviewers) {
    sum += reviewer.Review.rating;         // .Review
  }
  const average = sum / book.Reviewers.length;   // .length
  console.log(`${book.title} average rating: ${average}`);
}

// Average rating given by each member
const members = await LibraryMember.findAll({
  include: { model: LibraryBook, as: 'ReviewedBooks' },
});

for (const member of members) {
  let sum = 0;
  for (const book of member.ReviewedBooks) {   // .ReviewedBooks
    sum += book.Review.rating;
  }
  // average calc omitted for brevity
}
```

Exercise 9 – Library REST API Core

Goal: Wire up Express and the models into an API.

📁 **Create: controllers/libraryController.js, routes/libraryRoutes.js, server.js**

Q1 – Wire Up the Express App (server.js)

```
import express from 'express';
import { sequelize } from './models/index.js';
import libraryRoutes from './routes/libraryRoutes.js';

const app = express();
app.use(express.json());          // json()
app.use('/', libraryRoutes);
await sequelize.sync({ force: true });   // sync()
```

Q2 – Implement the Endpoints

In `controllers/libraryController.js`:

```
import { LibraryMember, LibraryBook } from '../models/index.js';

// POST /loans?memberId=1&bookId=1
export async function borrowBook(req, res) {
  const { memberId, bookId } = req.query;   // query

  try {
    const member = await LibraryMember.findByPk(memberId);
    const book   = await LibraryBook.findByPk(bookId);

    await member.addBorrowedBooks(book, {                                  // addBorrowedBooks
      through: { borrowedAt: new Date(), returnedAt: null },
    });

    res.status(201).json({ message: 'Book borrowed' });      // 201
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
}
```

Wire it up in `routes/libraryRoutes.js`:

```
import express from 'express';
import { borrowBook } from '../controllers/libraryController.js';

const router = express.Router();
router.post('/loans', borrowBook);   // post

export default router;
```